Git Cheat Sheet

TerminalVibes — The Terminal for Vibe Coders · neovand.github.io/terminalvibes

Orientation

```
pwd
```

Print the directory you are standing in

The full absolute path from / down to here. Half of all terminal confusion is being in the wrong folder — when anything misbehaves, pwd first.

```
ls
```

List the files and folders here

Names only by default. Give it a path (ls src) to peek inside a folder without moving into it.

```
ls -a
```

List everything, including hidden dotfiles

Files whose names start with a dot (.bashrc, .git, .env) are hidden by default — -a reveals them, plus the special . and .. entries.

```
ls -l
```

Long listing: permissions, owner, size, date

One file per line with the full story. Combine flags freely: ls -la shows the long view including hidden files.

```
whoami
```

Print your username

A quick identity check — most useful after sudo, over ssh, or when a script behaves as if it belongs to someone else.

```
clear
```

Wipe the screen (Ctrl+L does the same)

Only clears the view — nothing is undone or lost, and you can still scroll up. On Windows cmd the equivalent is cls, but in WSL and Git Bash clear works as shown.

```
man <command>
```

Open the full manual page for a command

Scroll with arrows or space, press q to quit the pager. Git Bash on Windows ships without man — use <command> --help there instead.

```
<command> --help
```

Quick usage summary and flag list

Faster than man when you just need a flag reminder. Great habit: run it on any command an AI suggests before running the real thing.

Navigation

```
cd <dir>
```

Move into a directory

Takes a relative path (cd projects) or an absolute one (cd /usr/local). Press TAB while typing — completion prevents the typos that cause most cd failures.

```
cd ..
```

Go up one level to the parent directory

Chain it to climb faster: cd ../.. goes up two levels in one hop.

```
cd
```

Jump home from anywhere (same as cd ~)

Home is /Users/you on macOS and /home/you on Linux. In WSL your Windows files live under /mnt/c — your Linux home is separate.

```
cd -
```

Bounce back to the previous directory

Toggles between your last two locations — perfect for hopping between a project and a log folder.

```
cd /
```

Go to the root of the filesystem

Everything on the machine lives somewhere beneath /. Look around with ls, but be careful running commands from here.

```
cd ~/projects
```

Go to a path spelled from your home folder

~ expands to your home directory, so this works no matter where you currently stand — the most reliable way to write personal paths.

Files & Folders

```
mkdir <name>
```

Create a new directory

Errors if it already exists — harmless, but mkdir -p stays quiet instead.

```
mkdir -p src/app/utils
```

Create nested directories in one go

-p makes every missing parent along the path and never complains if some already exist — the standard way to build a project skeleton.

```
touch <file>
```

Create an empty file (or update its timestamp)

If the file exists, only its modified time changes — the contents are untouched. Safe to run on anything.

```
cp <src> <dest>
```

Copy a file

If <dest> is a folder the copy lands inside it; if it is a filename you get a copy under that name. Overwrites silently — cp -i asks first.

```
cp -r <dir> <dest>
```

Copy a folder and everything inside it

Plain cp refuses directories; -r (recursive) descends into them. The classic quick backup: cp -r project project-backup.

```
mv <src> <dest>
```

Move a file — or rename it (same command)

mv old.txt new.txt renames; mv old.txt archive/ moves. It silently overwrites an existing destination — mv -i asks before clobbering.

```
rm <file>
```

Delete a file — permanently, no trash can

There is no undo and no recycle bin. Before deleting, ls the exact name you are about to remove; after, only backups or your editor's local history can help.

```
rm -r <dir>
```

Delete a folder and everything inside it

Recursive and permanent. Slow down when you see -rf: -f silences every warning. Wrong folder + wildcard + rm -rf is the classic terminal disaster.

```
rmdir <dir>
```

Delete a directory only if it is empty

The cautious cousin of rm -r — it refuses if anything is inside, which makes it a safe way to clean up folder skeletons.

```
open .
```

Open the current folder in Finder (macOS)

Linux: xdg-open . — WSL: explorer.exe . (note the dot). Handy bridge back to the GUI.

Viewing Files

```
cat <file>
```

Print a whole file to the screen

Best for short files. cat a huge or binary file and the screen floods — reach for less instead, and reset if the display garbles.

```
less <file>
```

Page through a big file comfortably

Space or arrows scroll, / searches forward, n repeats the search, q quits. man uses the same pager, so these keys work there too.

```
head <file>
```

Show the first 10 lines

head -n 25 shows 25. Perfect for checking what kind of file you are dealing with.

```
tail <file>
```

Show the last 10 lines

The end of a log file is usually where the newest — and most interesting — entries are.

```
tail -f server.log
```

Follow a file live as new lines arrive

The standard way to watch a running server write its log. It never exits on its own — Ctrl+C stops watching.

```
wc -l <file>
```

Count the lines in a file

wc alone prints lines, words and bytes. Counting is the quickest sanity check that a pipeline or filter did what you expected.

```
file <file>
```

Ask what kind of file this is

Reads the contents, not the extension — it will tell you a ".txt" is really a PNG. Useful before you cat something questionable.

Text & Pipes

```
<command> | <command>
```

Pipe: feed one command's output into the next

The superpower. Small tools compose into big answers: history | grep ssh finds every ssh command you have ever run.

```
<command> > out.txt
```

Redirect output into a file — OVERWRITES it

> truncates the file to zero before writing. If the file mattered, it is already gone — reach for >> when in doubt.

```
<command> >> out.txt
```

Append output to the end of a file

Adds without destroying — the safe default for collecting results, building logs, or writing a script line by line with echo.

```
<command> 2> errors.txt
```

Capture error output separately

Programs write normal output and errors to two different streams. 2> catches just the errors; 2>&1 merges them into one.

```
sort <file>
```

Sort lines alphabetically

sort -n sorts numerically (so 9 comes before 10), sort -r reverses. Sorting is also the required warm-up for uniq.

```
uniq -c
```

Collapse repeated adjacent lines and count them

Only removes duplicates that sit next to each other — which is why the recipe is always sort first, then uniq.

```
cut -d, -f1
```

Take one column from delimited text

-d sets the delimiter (a comma here), -f picks the field. cut -d: -f1 /etc/passwd lists every username.

```
sort | uniq -c | sort -rn
```

The classic frequency pipeline: count and rank

Sort to group, count the groups, sort the counts biggest-first. Top IPs in a log, most-used commands in history — same three steps every time.

Searching

```
grep "<text>" <file>
```

Print the lines that contain the text

The single most useful text tool. Quote the pattern so spaces and special characters survive the trip to grep.

```
grep -i "<text>" <file>
```

Search ignoring upper/lower case

ERROR, Error and error all match. Add -n to show line numbers with each hit.

```
grep -rn "<text>" .
```

Search every file under this folder, recursively

-r descends into subdirectories, -n prints file and line numbers — the terminal's project-wide find-in-files.

```
grep -v "<text>"
```

Keep the lines that do NOT match

The filter-out flag: cat server.log | grep -v DEBUG hides the noise so the signal stands out.

```
grep -c "<text>" <file>
```

Count matching lines instead of printing them

How many errors today? grep -c ERROR server.log answers with one number.

```
find . -name "*.md"
```

Find files by name pattern, anywhere below here

find searches file NAMES; grep searches file CONTENTS. Quote the pattern so the shell hands the wildcard to find instead of expanding it early.

```
find . -type d -name "node_modules"
```

Find directories by name

-type d matches directories only, -type f files only. Combine with -name for precise hunts.

Permissions

```
ls -l
```

Read the permission string on every file

The 10-character prefix decodes as type + rwx for user, group, other: -rwxr-xr-- means you can do anything, your group can read/run, others read only.

```
chmod +x <script>
```

Make a file executable

The fix for "Permission denied" when running your own script. After chmod +x, run it with ./script.sh.

```
chmod 755 <file>
```

Recipe: you rwx, everyone else read/run

The standard mode for scripts and directories. Each digit is one audience: user, group, other.

```
chmod 644 <file>
```

Recipe: you read/write, everyone else read-only

The standard mode for ordinary files — nobody can execute it, others cannot change it.

```
sudo <command>
```

Run one command as the administrator (root)

Respect it: sudo bypasses every safety rail. Never sudo a command you do not understand — especially one an AI wrote. Git Bash has no sudo; WSL does.

```
sudo !!
```

Re-run the previous command with sudo

!! expands to your last command — the classic move after "Permission denied" on something you meant to run as root.

Environment

```
echo $HOME
```

Print the value of a variable

$NAME expands to the variable's value before echo even runs. Try $USER, $SHELL, $PWD — the shell keeps its state in variables.

```
echo $PATH
```

The list of folders searched for commands

Colon-separated, checked left to right. "command not found" means the tool is not in any of these folders — or not installed at all.

```
which <command>
```

Print the full path of what would actually run

Answers "which python am I really running?" If which finds nothing, PATH is why your command is not found.

```
export VAR=value
```

Set a variable for this session and its children

Lasts until the terminal closes. To make it permanent, add the export line to ~/.bashrc or ~/.zshrc.

```
env
```

List every environment variable

The full picture of what programs inherit from your shell. Pipe it through grep to find one: env | grep PATH.

```
alias gs='git status'
```

Create a shortcut command

Type gs, get git status. Aliases live only in this session until you add them to your shell config file.

```
source ~/.bashrc
```

Reload your shell config without reopening the terminal

Run it after editing your config so changes take effect now. zsh users (macOS default): source ~/.zshrc.

Scripting

```
#!/usr/bin/env bash
```

Shebang: the required first line of a bash script

Tells the system which interpreter runs the file. A script is just saved commands — everything you type interactively works inside one.

```
chmod +x script.sh
```

Make the script runnable

One-time step per script. Without it you get “Permission denied” even on a file you wrote yourself.

```
./script.sh
```

Run a script from the current directory

The ./ is required — the current directory is deliberately NOT on PATH, so you must point at the script explicitly.

```
bash script.sh
```

Run a script without the executable bit

Hands the file straight to bash, skipping chmod. Handy for quick tests of a script you just wrote.

```
echo "$1"
```

Use the first argument passed to your script

$1, $2, … hold the arguments; $@ is all of them. Quote them so arguments containing spaces stay whole.

```
echo $?
```

Exit code of the last command — 0 means success

Every command reports success (0) or failure (anything else) as it finishes. Scripts, && and || all make decisions from this number.

```
<command> && <command>
```

Run the second only if the first SUCCEEDS

npm test && npm run deploy deploys only on green tests. Beware the classic horror: if cd fails, the next command runs in the wrong folder — && stops that.

```
<command> || <command>
```

Run the second only if the first FAILS

The fallback operator: npm test || echo "tests failed" — the right-hand side is the plan B.

```
<command> ; <command>
```

Run both, one after the other, regardless

No safety logic at all — the second runs even if the first exploded. Prefer && when the second step depends on the first.

Panic Button

```
Ctrl+C
```

Cancel the running command — or discard a typed line

Typed a scary command? Just don’t press Enter — Ctrl+C throws the line away unrun. Nothing you type executes until you press Enter.

```
q
```

Quit a full-screen pager (less, man, git log)

A “frozen” terminal is very often just a pager waiting politely. If q does nothing, try Ctrl+C first, then q.

```
:q!
```

Trapped in vim? Press Esc, then type :q! and Enter

Quits without saving. Some commands open vim as an editor without warning — this is the universal exit.

```
Ctrl+D
```

Close the shell (same as typing exit)

Sends “end of input”. If a program is waiting for input you did not intend to give, Ctrl+D on an empty line often releases it.

```
reset
```

Fix a garbled, glitchy terminal display

After cat-ing a binary file the screen can fill with junk — reset repaints everything. It takes a second; your session survives.

```
echo rm *.log
```

Preview what a wildcard will hit — before deleting

echo expands the glob harmlessly and shows the exact file list rm would receive. Make this reflex and rm loses most of its teeth.

```
rm
```

Deleted a file? There is no undo — but check your editor

rm skips the trash entirely. VS Code’s Timeline view keeps local history of files you edited, and if the project uses git, a committed file can be restored.

```
history
```

What did I actually run? Read the record

Numbered list of your recent commands — the first step of any post-incident investigation, and Ctrl+R searches it interactively.